

Master's in Asset Management Orientation BootCamp

AI Tools: Introduction to Claude Code

Session Information

Instructor: Xugan Chen
Email: xugan.chen@yale.edu
Meeting time / location: TBD.
Duration: 3 hours, one 10-minute break
Assessment: None. No homework, no grading.
Course materials: <https://github.com/xuganchen/BootCamp-Introduction-to-Claude-Code>

Overview

This is a hands-on, finance-applied working session that introduces students to AI-assisted workflows for finance and data analysis. The goal is to move students from "I have Claude Code installed" to "I can delegate a bounded asset-management task to it and verify the result is correct."

The session builds on the pre-work (DataCamp modules and the Paul Goldsmith-Pinkham series). It opens with a short introduction to AI-assisted workflows and Claude Code, then turns to an in-class mini-project that applies agentic coding to a real asset-management task: building a private credit dataset from raw SEC filings and analyzing it. The project runs in four steps, and each step introduces the concept it needs (plan mode and sub-agents, skills, grounding and retrieval) at the point the work demands it, rather than as a separate lecture. From there we generalize those patterns beyond the project and cover AI safety. It closes with a few advanced-usage examples, most of them my own projects, to show what is possible beyond the basics.

Prerequisite

Have a terminal agent installed and running before the session: we will work and experiment on the mini-project described below together. [SETUP.md](#) in the course repository walks through it in about ten minutes and gives two options, one of them free for students. Claude Code is what will be on the screen; Google's Antigravity CLI also works.

Setup instructions: <https://github.com/xuganchen/BootCamp-Introduction-to-Claude-Code/blob/main/SETUP.md>

Session Outline

1. Introduction

- Framing: how to apply AI, specifically Claude Code, in a daily finance research workflow.

- The core loop: giving instructions, reviewing changes, and iterating – and why reviewing is the hard part.
- Basics: setup, tokens, and the context window.

2. Applied AM Project: Building a Private Credit Dataset from SEC Filings

An in-class mini-project that builds a private credit research dataset from raw SEC filings. Each student picks one publicly traded business development company (BDC) and constructs two panels from its 10-K and 10-Q filings.

- **Dataset:** SEC EDGAR filings. Free, public, and requiring no credentials or licensing. You pick your own BDC, and parse from 2023-01-01 onward. A reference panel is released at Step 3, so the analysis works regardless of how far your own parser gets.
- **Deliverable, identical for everyone:** a *BDC-quarter* panel (fund level: NAV, leverage, yield, non-accruals, portfolio size) and a *BDC-quarter-investment* panel (position level: borrower, seniority, spread, maturity, cost, fair value), both built from the Schedule of Investments.

Nothing executable is provided: you write the downloader, the parser and the verification check yourself, with the agent. That is the session. What you get instead is the target schema, the EDGAR ground rules, every prompt used in class, and the reference panels.

The project runs in four steps. Each step introduces the concept it needs, at the point the work demands it.

- **Step 0. Naive baseline.** One short prompt, run three times across models and effort levels. All three invent a balance-sheet tie-out unasked and pass it, the three schemas cannot be pooled, and one of them is broken in a way its own passing check cannot see. *Concept:* a passing check is only as good as its scope, and you did not choose that scope.
- **Step 1. Write a spec, then build.** Replace the one-liner with a real specification (objective, deliverables, folder structure, schema, acceptance criteria) and let the agent run. *Concepts:* plan mode; sub-agents used for independent verification, where one writes the parser and a second writes the test without seeing it.
- **Step 2. Generalize across quarters.** Turn the working parser into a reusable skill and run it across every quarter the BDC has filed, using the tie-out as an automated test that shows exactly where it broke. *Concepts:* skills; coverage as the deliverable.
- **Step 3. Analysis and reporting.** Sample coverage, summary statistics, time series, deal terms, and cross section, written up in a style you supply rather than one the model invents. *Concepts:* grounding; analysis as a verification layer of its own.

Why this dataset: private credit is the fastest-growing corner of asset management, BDC filings are its only public window, and the filing verifies your own work. The Schedule of Investments must foot to the balance sheet, so the document tells you whether your parse is right.

3. More Useful Concepts

With the project having made the difficulties concrete, we generalize beyond it.

- **Working tips:**
 1. plan before acting;
 2. be specific;
 3. always have a backup;
 4. manage your context window;
 5. always have a way to verify outputs;
 6. ask the model to take notes;
 7. be replicable;
 8. stay organized with your code, data, and notes.
- **Transfer:** the same plan mode, skills, and retrieval patterns applied to other work, such as writing, literature review, and exam preparation.
- **AI safety:** permissions and dev containers as the real alternative.

4. Advanced Usage Examples

A sense of what is possible beyond the basics, time permitting:

- **Personalized “brain”:** indexed retrieval over a corpus you own, picking up where Step 3’s grounding left off (GBrain, by Garry Tan, <https://github.com/garrytan/gbrain>).
- **Personal database management:** *TravelTab*, a self-hosted Amtrak ticket tracker that auto-imports tickets from email, exports structured records to a searchable dashboard, syncs to a calendar, and watches fares on a schedule. The ingest is this session’s project in miniature: messy documents in, structured records out.
- **The same stack in another domain:** *PetTab*, a pet-health logger built on TravelTab’s architecture, with an agent embedded in the product that answers questions over a read-only snapshot of the database.
- **An agent with an address:** an agent in a Linux container, reached from WhatsApp, Slack, Telegram or email (NanoClaw, <https://github.com/nanocoai/nanoclave>).

Resources

Additional resources are collected on the instructor’s website (<https://www.xuganchen.com/public-goods#working-with-ai-tools-for-economicsfinance-research>), including:

- Using LLMs with Cursor: Modern AI for Economics Research by Benjamin Golub, Markus Academy
- A Series on Claude Code by Paul Goldsmith-Pinkham
- Thinking with Agents by UVM Economics BootCamp
- Agent Skills with Anthropic by DeepLearning.AI
- Finance Skill Plugins by Anthropic